

VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software

Unit Coverage: 1 – 5

Max Marks: 100

Chosen Software: GIT

Student Name	Registration Number
Arnima Awasthi	24BAI10813
Slot	Date of Submission
A24	31-03-2026

1. Introduction

A vast portion of the software used today was not built by a single entity, It was initialized by somebody and then put up on the internet for people to come together, use it and also help developing it, basically a community of developers working together. This is what open- source software is all about. It allows people to share their work, get feedback, improve it and allow people to use it freely. Many programming languages and applications are in fact open source, or they were at some point of time.

In this project, I have chosen Python as my open-source software. Python is a high-level, interpreted programming language.It is an open-source software that is developed and maintained by a community of developers. It's source code is open for everybody to download, read and interact with on systems like Linux or Windows.

Python was originally created to solve a specific programming language problem, something powerful to handle complex tasks but at the same time simple enough for humans to understand and code in it. It was built to bridge the gap between high-level languages and basic shell scripting. Over time, it became very popular because of its easy to understand nature and shorter and quicker codes than other available languages .Since it is open source, developers around the world contribute to improving it.

Today, Python serves as the foundation for modern engineering, Artificial Intelligence, Data Science and Web Development.

In this report, I will study Python from different perspectives such as its origin, license, working in Linux, and its role in the open-source ecosystem. I will also compare it with proprietary software.

Along with the report, I have created five shell scripts to demonstrate basic Linux commands and automation. These scripts helped me understand how open-source tools work in practice.

2. Choosing Your Software

Your choice	Write your chosen software here before submission: Python
--------------------	---

Part A — Origin and Philosophy (Units 1 & 2)

A1. Origin Story

Coming to how Python actually came to life, we need to go way back. It all started in 1980s, Guido Van Rossum, was working at the CWI(Centrum Wiskunde & Informatica)in the Netherlands as a researcher and programmer, he and his team was working on a project called Amoeba, but those days, programmers had fewer choices,such as either working on C, which was powerful and fast but, complex to understand and time-consuming to write or use shell scripts which were great for quick automation but struggled with complex logic.

Rossum also faced a challenge, He realized there was gap and he wanted a language that was as easy to understand as the English language but at the same time as efficient as other coding languages available. He was working with ABC at that time, but was wanting to build something better.

During the Christmas holidays of 1989, things changed, Rossum, gave his hobby a chance, he wanted to keep himself occupied and what better than a hobby. He started working on python as fun, the name PYTHON, resembling the snake doesn't actually come form that, Rossum actually named it after his favourite program at that time, Monty Python's Flying Circus, telling us that it indeed was a fun project for him that is now doing work, worth millions.

He developed Python as he wanted, easier syntax, similar to English but at the same time powerful and efficient enough to handle complex coding.

Instead of keeping Python private or for monetary profits, Rossum released it as open source. This allowed developers all over the world to use and improve it. It is a language that is developed by an entire community instead of a corporate board. It is free and available to anybody for use. It was basically a tool built by developers for the developers.

Since it's development in 1989, Python has come a long way, various versions of it have been released, with upgrades and modifications over the years. It has a huge number of in-built functions and libraries and plays a huge role in the development of the Artificial Intelligence, we all use in our day-to-day lives.

This story not only tells us about how python was developed but also tells us how, things done as just hobbies or for fun and came have huge impacts on not only our lives but also others.

A2. License: what it actually says

Python is released under the Python Software Foundation License (PSFL), which is a permissive open-source license. This means it gives a lot of freedom to users without many restrictions.

The idea of open source is based on four main freedoms:

1. The freedom to run the program for any purpose
2. The freedom to study how the program works
3. The freedom to modify the program
4. The freedom to share the program with others

The PSFL supports all these freedoms and actually allows the developers to use Python in various ways.

The PSF License is a very permissive License, it is very flexible and allows others to use it for closed-source softwares, because of this, it is popular in many companies like Google and Netflix for their

systems. They are not forced to share their private code but they still contribute to improving Python because it benefits them.

Contrary to the GPL License, which is stricter and one would have to release their code changes back to the public to use it in their closed-source softwares. So, Python's creators used a more relaxed path to encourage as much adoption as possible

Overall, this permissive license helped Python grow quickly and become a widely used language.

But, Everything does come with a price, The PSF License says that if the software crashes or causes an error, the developers are not legally responsible, well, I guess that is the price we pay for getting such an amazing software for free.

A3. The ethics of open source

Open-source software raises an interesting question: should all software be open source?

On one hand, open source promotes transparency and sharing of knowledge. Anyone can check the code, which makes it more secure. It also helps students and developers learn from real projects. Not all companies can afford to make their software open source. Some projects require a lot of investment, and companies need profit to sustain themselves.

Companies like Google and Netflix also use open-source software like Linux. Some people feel this is unfair, but these companies also contribute back to the community.

The idea of "standing on the shoulders of giants" is very important here. Developers today do not start from scratch. They build on existing tools like Python, Linux, and Git.

In my opinion, open source supports innovation rather than limiting it. It saves time and allows developers to focus on new ideas instead of reinventing things Python follows a simple philosophy known as the "Zen of Python," which includes ideas like "Simple is better than complex" and "Readable code is better than complicated code."

The main ethical idea behind Python and open source is that knowledge should be shared. When tools and software are freely available-more people can learn, improve, and build new things.

This idea is often described as "standing on the shoulders of giants." It means that developers don't have to start from scratch. They can use existing tools and build something better on top of them. For example, a student can use Python libraries created by others to develop new applications without spending years building basic tools.

Python also had an interesting leadership model. For many years, Guido van Rossum was known as the "Benevolent Dictator for Life," which means he had the final say in decisions. This helped keep the language consistent and avoided confusion.

Later, in 2018, he stepped down, and Python started being managed by a group called the Steering Council. This shows that Python has become a community-driven project where many people contribute and make decisions together.

Part B — Linux Footprint (Unit 2)

B1. Installation and Package Management

The first step in auditing Python on a Linux system is understanding how it is installed. Unlike Windows, where software is usually downloaded from websites, Linux uses a package manager.

To install Python, I used the following commands:

```
sudo apt update
sudo apt install python3 -y
```

Here, apt is the package manager used in Ubuntu. It installs software from trusted repositories. This means the software is verified and maintained by the Linux community, which makes it more secure compared to downloading random files from the internet.

This shows an important concept of open source: users can trust the system because the process is transparent and controlled.

B2. Mapping the Filesystem Hierarchy

To understand where Python is located in the system, I used different commands to find its files.

First, I located the main executable file using:

```
which python3
```

The output was:

```
/usr/bin/python3
```

This is the binary file, which means it is the main program that runs Python.

Next, I looked at where Python libraries are stored. These libraries contain important modules that Python uses:

```
/usr/lib/python3
```

I also found header files in:

```
/usr/include/python3.x
```

These are used when Python interacts with lower-level system components like C programs.

From this, we can see that Python follows the Linux Filesystem Hierarchy Standard and is properly organized within the system directories.

B3. Process and Memory Inspection

When a Python program runs, it becomes a process in the system. To observe this, I ran a Python script and used the following command:

```
ps aux | grep python3
```

This command shows all running Python processes along with details like CPU usage and memory usage.

This step helps us understand how Python uses system resources. One advantage of open-source systems like Linux is that users can easily monitor what is happening in the system. This level of transparency is very useful for debugging and performance analysis.

B4. Permissions and User Environment

In Linux, every file has permissions that control who can read, write, or execute it.

To check the permissions of Python, I used:

```
ls -l /usr/bin/python3
```

The output showed:

- Owner: root
- Permissions: `-rwxr-xr-x`

This means:

- The owner (root) has full permissions
- Other users can read and execute the file but cannot modify it

This is important for system security. Even though all users can use Python, they cannot change the main system files. This prevents unauthorized modifications and keeps the system stable.

B5. Versioning and Environment Audit

Open-source software is updated frequently, and different projects may require different versions.

To check the installed version of Python, I used:

```
python3 --version
```

To see installed Python packages, I used:

```
pip3 list
```

Python uses a package manager called **pip**, which allows users to install and manage additional libraries.

This makes it easy to keep track of all installed packages and their versions. Compared to proprietary software, this level of visibility is very useful because users know exactly what is installed in their system.

Part C — The FOSS Ecosystem (Units 3 & 4)

C1. Dependencies: What Python Relies On

Python does not work alone. It depends on several other open-source tools to function properly on a Linux system.

One important dependency is the **GNU C Library (glibc)**. Since Python is mainly written in C, it uses this library for basic tasks like handling files and memory.

Another important dependency is **OpenSSL**, which is used for secure communication over the internet. For example, when Python downloads packages, encryption is handled using OpenSSL.

Python also includes **SQLite**, which is a lightweight database system. This allows developers to store and manage data without installing a large database like MySQL.

If Python is built from source code, it also requires tools like **GCC (GNU Compiler Collection)** and **Make**, which are part of the GNU toolchain.

This shows that Python is part of a larger ecosystem and depends on many open-source projects to work efficiently.

C2. What Python Has Enabled

Python is often called a “glue language” because it helps connect different technologies and tools.

It has played a major role in many important fields. In data science and artificial intelligence, libraries like NumPy, Pandas, and TensorFlow are built using Python. These tools have made Python very popular in AI and machine learning.

In web development, frameworks like Django and Flask allow developers to build websites easily and quickly.

Python is also widely used for automation. Many system tools in Linux are written in Python because it is simple and easy to understand.

Overall, Python has helped many open-source projects grow and has become an important part of modern technology.

C3. The LAMP Stack Connection

The LAMP stack stands for Linux, Apache, MySQL, and PHP. Traditionally, PHP was used as the main programming language in this stack.

However, in modern systems, Python is often used instead of PHP.

In this setup, Python works as the logic layer. It processes data from the MySQL database and sends it to the user through the Apache web server.

Since all these components are open source, they work well together. Python can easily connect with databases and web servers using standard protocols.

This shows how open-source tools can be combined to build powerful systems.

C4. Community and Governance

Python is not controlled by a single company. It is managed by the **Python Software Foundation (PSF)**, which is a non-profit organization.

The development of Python is very open. New features and changes are discussed through **Python Enhancement Proposals (PEPs)**. Anyone in the community can suggest improvements.

There are also many ways for developers to connect and learn. Events like **PyCon** bring together Python users from around the world.

In addition, there are online forums and chat platforms where beginners can ask questions and get help from experienced developers.

This strong community support is one of the main reasons why Python continues to grow and improve.

Part D — Open Source vs Proprietary (Critical Analysis)

D1. Comparison Table: Python vs MATLAB

The following table compares Python (open source) with MATLAB (proprietary software):

Dimension	Open Source	Proprietary Alternative
<i>Cost</i>	Free to use, download and distribute.	Requires paid license, often expensive.
<i>Security (who can audit the code?)</i>	Source code is open, anyone can check it.	Source code is closed. Only company can access.
<i>Support and reliability</i>	Community support (forums, GitHub, Stack Overflow)	Official support from MathWorks
<i>Freedom to modify</i>	Users can modify and customize freely	Users cannot modify the core software

D2. In-Depth Analysis

MATLAB provides a well-designed and user-friendly environment with professional support, which can be helpful for beginners and organizations.

However, Python has several important advantages. One of the main benefit is that, it is free. In MATLAB, users must pay for licenses, and if they stop paying, they may lose access to the software. In contrast, Python is always available and does not depend on any company.

Another important difference is security and transparency. In Python, the source code is open, so developers around the world can check and fix issues. In MATLAB, users must depend on the company to identify and fix problems.

At the same time, MATLAB's centralized control can make it more consistent and easier to manage. On the other hand, Python has a very large number of libraries, which can sometimes be confusing for beginners.

D3. Overall Verdict

Based on this audit, I would recommend Python for real-world use, especially in areas like data science, web development, and research.

The fact that Python is free makes it accessible to students, startups, and developers everywhere. It also has a large number of libraries, which makes it very flexible and useful for many different applications.

I would also be interested in contributing to the Python ecosystem. Since it is open source, even small contributions like fixing bugs or improving documentation can help many users.

Being part of an open-source project like Python allows developers to learn, collaborate, and contribute to something that benefits people all over the world.

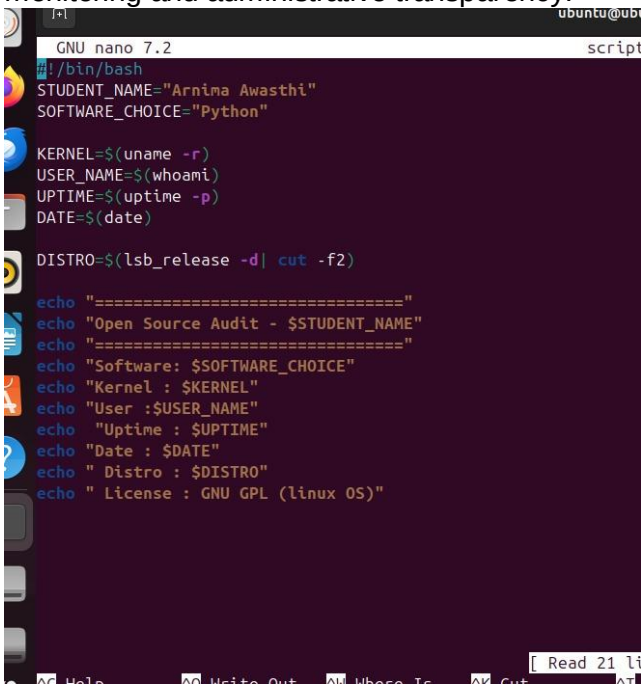
Conclusion

From my perspective, Python is a good choice for most developers because it is free, flexible, easy to understand and efficient. It allows full control over the code and encourages collaboration. While proprietary tools may provide dedicated support, they are often costly and restrictive in terms of usage and customization.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

This script basically serves as a System Identity Report, which functions like a welcome screen to display the important Linux environment details. It uses core shell scripting concepts like variables and command substitution `$()` to capture and store output from standard tools like `uname`, `uptime`, `whoami` and `hostnamectl`. By automating the kernel version, current user, system uptime and distribution name, the script demonstrates how basic Linux commands can be used for system monitoring and administrative transparency.

A screenshot of a terminal window with the Ubuntu desktop environment visible on the left. The terminal is running the nano text editor, editing a file named 'script1.sh'. The script content is as follows:

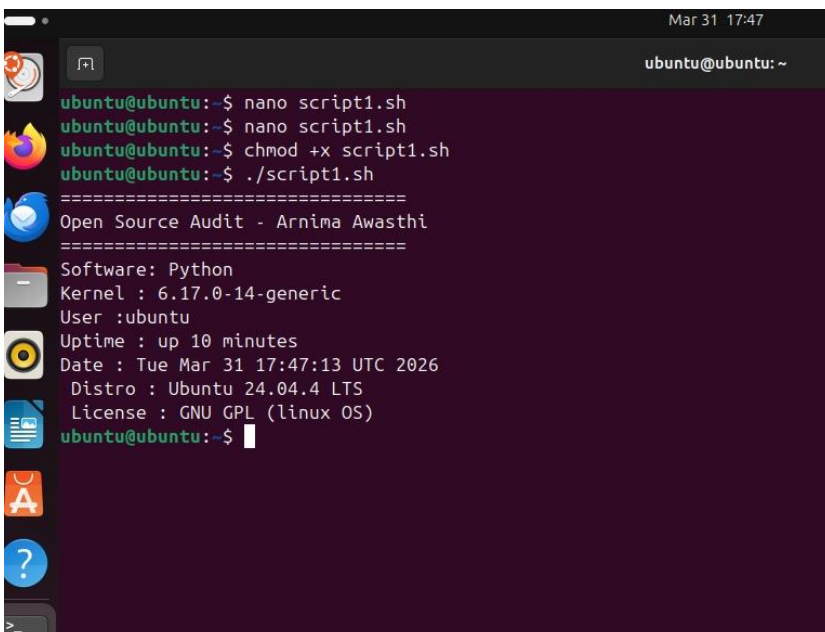
```
#!/bin/bash
STUDENT_NAME="Arnima Awasthi"
SOFTWARE_CHOICE="Python"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)

DISTRO=$(lsb_release -d | cut -f2)

echo "=====
echo "Open Source Audit - $STUDENT_NAME"
echo "=====
echo "Software: $SOFTWARE_CHOICE"
echo "Kernel : $KERNEL"
echo "User :$USER_NAME"
echo "Uptime : $UPTIME"
echo "Date : $DATE"
echo " Distro : $DISTRO"
echo " License : GNU GPL (linux OS)"
```

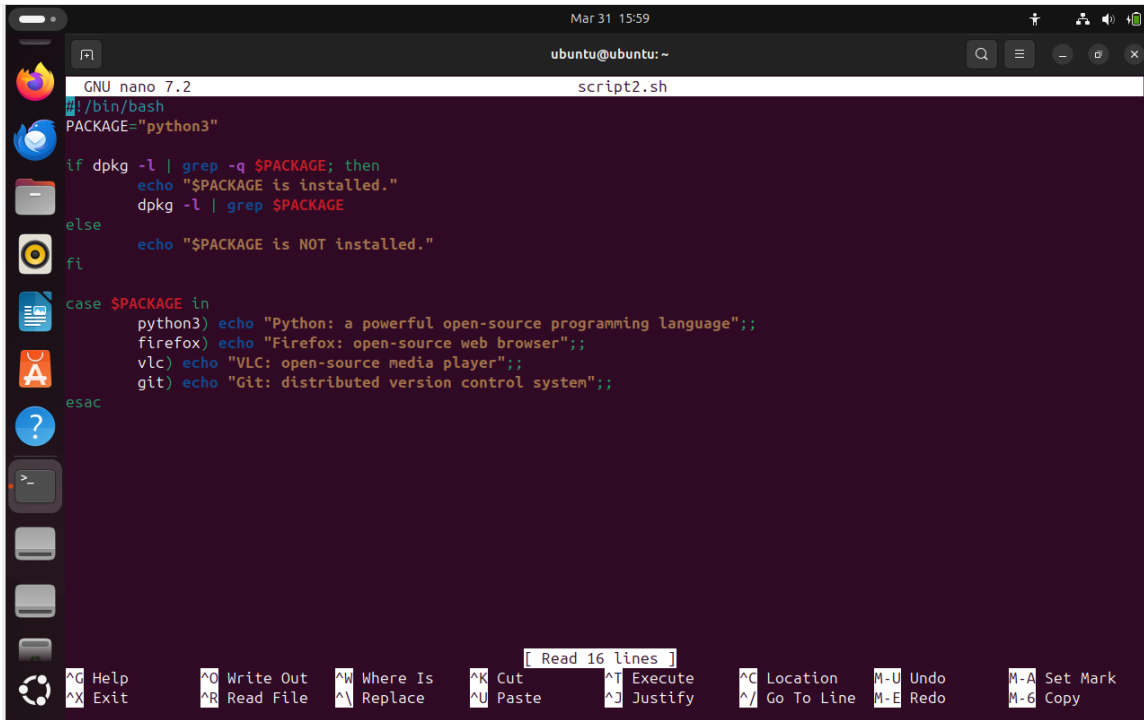
OUTPUT:

A screenshot of a terminal window showing the execution of the script1.sh file. The terminal output is as follows:

```
ubuntu@ubuntu:~$ nano script1.sh
ubuntu@ubuntu:~$ nano script1.sh
ubuntu@ubuntu:~$ chmod +x script1.sh
ubuntu@ubuntu:~$ ./script1.sh
=====
Open Source Audit - Arnima Awasthi
=====
Software: Python
Kernel : 6.17.0-14-generic
User :ubuntu
Uptime : up 10 minutes
Date : Tue Mar 31 17:47:13 UTC 2026
Distro : Ubuntu 24.04.4 LTS
License : GNU GPL (linux OS)
ubuntu@ubuntu:~$
```

Script 2 — FOSS Package Inspector (Unit 2)

In order to confirm the installation of an open-source software package and obtain its metadata, this script serves as a Package Inspector. The script determines whether the software is present on the system by using conditional if-else logic in conjunction with the dpkg and grep commands. Additionally, it uses a case statement for pattern matching, offering a unique explanation of the function and philosophy of the package. This script shows how shell scripting can automate software environment auditing and provides an example of practical Linux package management.



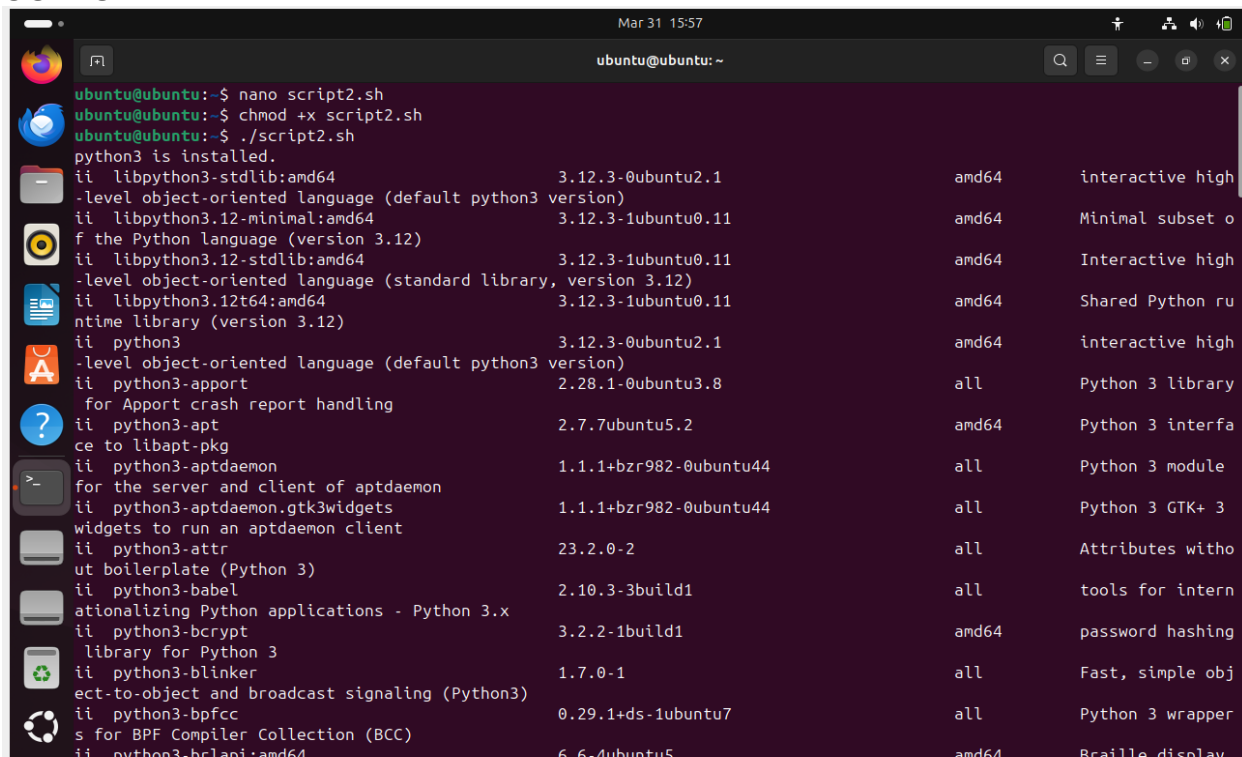
```

GNU nano 7.2 script2.sh
#!/bin/bash
PACKAGE="python3"

if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -l | grep $PACKAGE
else
    echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
    python3) echo "Python: a powerful open-source programming language";;
    firefox) echo "Firefox: open-source web browser";;
    vlc) echo "VLC: open-source media player";;
    git) echo "Git: distributed version control system";;
    esac
  
```

OUTPUT:



```

ubuntu@ubuntu:~$ nano script2.sh
ubuntu@ubuntu:~$ chmod +x script2.sh
ubuntu@ubuntu:~$ ./script2.sh
python3 is installed.
ii libpython3-stdlib:amd64 3.12.3-0ubuntu2.1 amd64 interactive high
-level object-oriented language (default python3 version)
ii libpython3.12-minimal:amd64 3.12.3-1ubuntu0.11 amd64 Minimal subset o
f the Python language (version 3.12)
ii libpython3.12-stdlib:amd64 3.12.3-1ubuntu0.11 amd64 Interactive high
-level object-oriented language (standard library, version 3.12)
ii libpython3.12t64:amd64 3.12.3-1ubuntu0.11 amd64 Shared Python ru
ntime library (version 3.12)
ii python3 3.12.3-0ubuntu2.1 amd64 interactive high
-level object-oriented language (default python3 version)
ii python3-apport 2.28.1-0ubuntu3.8 all Python 3 library
for Apport crash report handling
ii python3-apt 2.7.7ubuntu5.2 amd64 Python 3 interfa
ce to libapt-pkg
ii python3-aptdaemon 1.1.1+bzr982-0ubuntu44 all Python 3 module
for the server and client of aptdaemon
ii python3-aptdaemon.gtk3widgets 1.1.1+bzr982-0ubuntu44 all Python 3 GTK+ 3
widgets to run an aptdaemon client
ii python3-attr 23.2.0-2 all Attributes witho
ut boilerplate (Python 3)
ii python3-babel 2.10.3-3build1 all tools for intern
ationalizing Python applications - Python 3.x
ii python3-bcrypt 3.2.2-1build1 amd64 password hashing
library for Python 3
ii python3-blinker 1.7.0-1 all Fast, simple obj
ect-to-object and broadcast signaling (Python3)
ii python3-bpfc 0.29.1+ds-1ubuntu7 all Python 3 wrapper
s for BPF Compiler Collection (BCC)
ii python3-brlapi:amd64 6.6-4ubuntu5 amd64 Braille display
  
```

ubuntu@ubuntu: ~				
ii python3-certifi	2023.11.17-1	all	root certificate	
s for validating SSL certs and verifying TLS hosts (python3)				
ii python3-cffi-backend:amd64	1.16.0-2build1	amd64	Foreign Function	
Interface for Python 3 calling C code - runtime				
ii python3-chardet	5.2.0+dfsg-1	all	Universal Charac	
ter Encoding Detector (Python3)				
ii python3-click	8.1.6-2	all	Wrapper around o	
ptparse for command line utilities - Python 3.x				
ii python3-colorama	0.4.6-4	all	Cross-platform c	
olored terminal text in Python - Python 3.x				
ii python3-commandnotfound	23.04.0	all	Python 3 binding	
s for command-not-found.				
ii python3-configobj	5.0.8-3	all	simple but power	
ful config file reader and writer for Python 3				
ii python3-cryptography	41.0.7-4ubuntu0.1	amd64	Python library e	
xposing cryptographic recipes and primitives (Python 3)				
ii python3-cups:amd64	2.0.1-5build6	amd64	Python3 bindings	
for CUPS				
ii python3-cupshelpers	1.5.18-1ubuntu9	all	Python utility m	
odules around the CUPS printing system				
ii python3-dateutil	2.8.2-3ubuntu1	all	powerful extensi	
ons to the standard Python 3 datetime module				
ii python3-dbus	1.3.2-5build3	amd64	simple interproc	
ess messaging system (Python 3 interface)				
ii python3-debconf	1.5.86ubuntu1	all	interact with de	
bconf from Python 3				
ii python3-debian	0.1.49ubuntu2	all	Python 3 modules	
to work with Debian-related data formats				
ii python3-defer	1.0.6-2.1ubuntu1	all	Small framework	
for asynchronous programming (Python 3)				
ii python3-distro	1.9.0-1	all	Linux OS platfor	
m information API				
ii python3-distro-info	1.7build1-1.500	all	information abou	
ubuntu@ubuntu: ~				
ii python3-pexpect	4.9-2	all	Python 3 module	
for automating interactive applications				
ii python3-pil:amd64	10.2.0-1ubuntu1	amd64	Python Imaging L	
ibrary (Python3)				
ii python3-pkg-resources	68.1.2-2ubuntu1.2	all	Package Discover	
y and Resource Access using pkg_resources				
ii python3-problem-report	2.28.1-0ubuntu3.8	all	Python 3 library	
to handle problem reports				
ii python3-ptyprocess	0.7.0-5	all	Run a subprocess	
in a pseudo terminal from Python 3				
ii python3-pygments	2.17.2+dfsg-1	all	syntax highlight	
ing package written in Python 3				
ii python3-pyparsing	3.1.1-1	all	alternative to c	
reating and executing simple grammars - Python 3.x				
ii python3-pyrsistent:amd64	0.20.0-1build2	amd64	persistent/funct	
ional/immutable data structures for Python				
ii python3-requests	2.31.0+dfsg-1ubuntu1.1	all	elegant and simp	
le HTTP library for Python3, built for human beings				
ii python3-rich	13.7.1-1	all	render rich text	
, tables, progress bars, syntax highlighting, markdown and more				
ii python3-serial	3.5-2	all	pyserial - modul	
e encapsulating access for the serial port				
ii python3-six	1.16.0-4	all	Python 2 and 3 c	
ompatibility library				
ii python3-software-properties	0.99.49.3	all	manage the repos	
itories that you install software from				
ii python3-speechd	0.12.0-rc2-2build3	all	Python interface	
to Speech Dispatcher				
ii python3-sss	2.9.4-1.1ubuntu6.3	amd64	Python3 module f	
or the System Security Services Daemon				
ii python3-systemd	235-1build4	amd64	Python 3 binding	
s for systemd				
ii python3-typing-extensions	4.10.0-1	all	Backported and F	

```
Mar 31 15:59
ubuntu@ubuntu: ~
ii python3-software-properties 0.99.49.3 all manage the repos
itories that you install software from
ii python3-speechd 0.12.0~rc2-2build3 all Python interface
to Speech Dispatcher
ii python3-sss 2.9.4-1.1ubuntu6.3 amd64 Python3 module f
or the System Security Services Daemon
ii python3-systemd 235-1build4 amd64 Python 3 binding
s for systemd
ii python3-typing-extensions 4.10.0-1 all Backported and E
xperimental Type Hints for Python
ii python3-tz 2024.1-2 all Python3 version
of the Olson timezone database
ii python3-uno 4:24.2.7-0ubuntu0.24.04.4 amd64 Python-UNO bridg
e
ii python3-update-manager 1:24.04.12 all Python 3.x modul
e for update-manager
ii python3-urllib3 2.0.7-1ubuntu0.6 all HTTP library wit
h thread-safe connection pooling for Python3
ii python3-wadllib 1.3.6-5 all Python 3 library
for navigating WADL files
ii python3-xdg 0.28-2 all Python 3 library
to access freedesktop.org standards
ii python3-xkit 0.5.0ubuntu6 all library for the
manipulation of xorg.conf files (Python 3)
ii python3-yaml 6.0.1-2build2 amd64 YAML parser and
emitter for Python3
ii python3.12 3.12.3-1ubuntu0.11 amd64 Interactive high
-level object-oriented language (version 3.12)
ii python3.12-minimal 3.12.3-1ubuntu0.11 amd64 Minimal subset o
f the Python language (version 3.12)
Python: a powerful open-source programming language
ubuntu@ubuntu:~$
```

```
Mar 31 15:59
ubuntu@ubuntu: ~
ii python3-software-properties 0.99.49.3 all manage the repos
itories that you install software from
ii python3-speechd 0.12.0~rc2-2build3 all Python interface
to Speech Dispatcher
ii python3-sss 2.9.4-1.1ubuntu6.3 amd64 Python3 module f
or the System Security Services Daemon
ii python3-systemd 235-1build4 amd64 Python 3 binding
s for systemd
ii python3-typing-extensions 4.10.0-1 all Backported and E
xperimental Type Hints for Python
ii python3-tz 2024.1-2 all Python3 version
of the Olson timezone database
ii python3-uno 4:24.2.7-0ubuntu0.24.04.4 amd64 Python-UNO bridg
e
ii python3-update-manager 1:24.04.12 all Python 3.x modul
e for update-manager
ii python3-urllib3 2.0.7-1ubuntu0.6 all HTTP library wit
h thread-safe connection pooling for Python3
ii python3-wadllib 1.3.6-5 all Python 3 library
for navigating WADL files
ii python3-xdg 0.28-2 all Python 3 library
to access freedesktop.org standards
ii python3-xkit 0.5.0ubuntu6 all library for the
manipulation of xorg.conf files (Python 3)
ii python3-yaml 6.0.1-2build2 amd64 YAML parser and
emitter for Python3
ii python3.12 3.12.3-1ubuntu0.11 amd64 Interactive high
-level object-oriented language (version 3.12)
ii python3.12-minimal 3.12.3-1ubuntu0.11 amd64 Minimal subset o
f the Python language (version 3.12)
Python: a powerful open-source programming language
ubuntu@ubuntu:~$
```

Script 3 — Disk and Permission Auditor (Unit 2)

This script operates as a Directory and Storage Auditor, automating the inspection of critical Linux system paths like /etc, /var/log, and /usr/bin. By employing a for loop and existence checks, the script iterates through these directories to retrieve disk usage data via the du command and permission metadata using ls -ld. It further utilizes text-processing tools like awk and cut to parse and format the output. This script demonstrates the power of shell automation for performing recursive system maintenance and security audits.

```

GNU nano 7.2 script3.sh
#!/bin/bash

DIRS="/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "-----"

for DIR in "${DIRS[@]};do
    if[-d "$DIR"];then
        PERMS=$(ls -ld $DIR| awk '{print $1,$3,$4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done
if[ -d"/usr/lib/python3"]; then
    echo"Python config exists"
    ls -ld/usr/lib/python3
fi

```

[Read 19 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-
^Y Exit ^P Read File ^L Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo M-

OUTPUT:

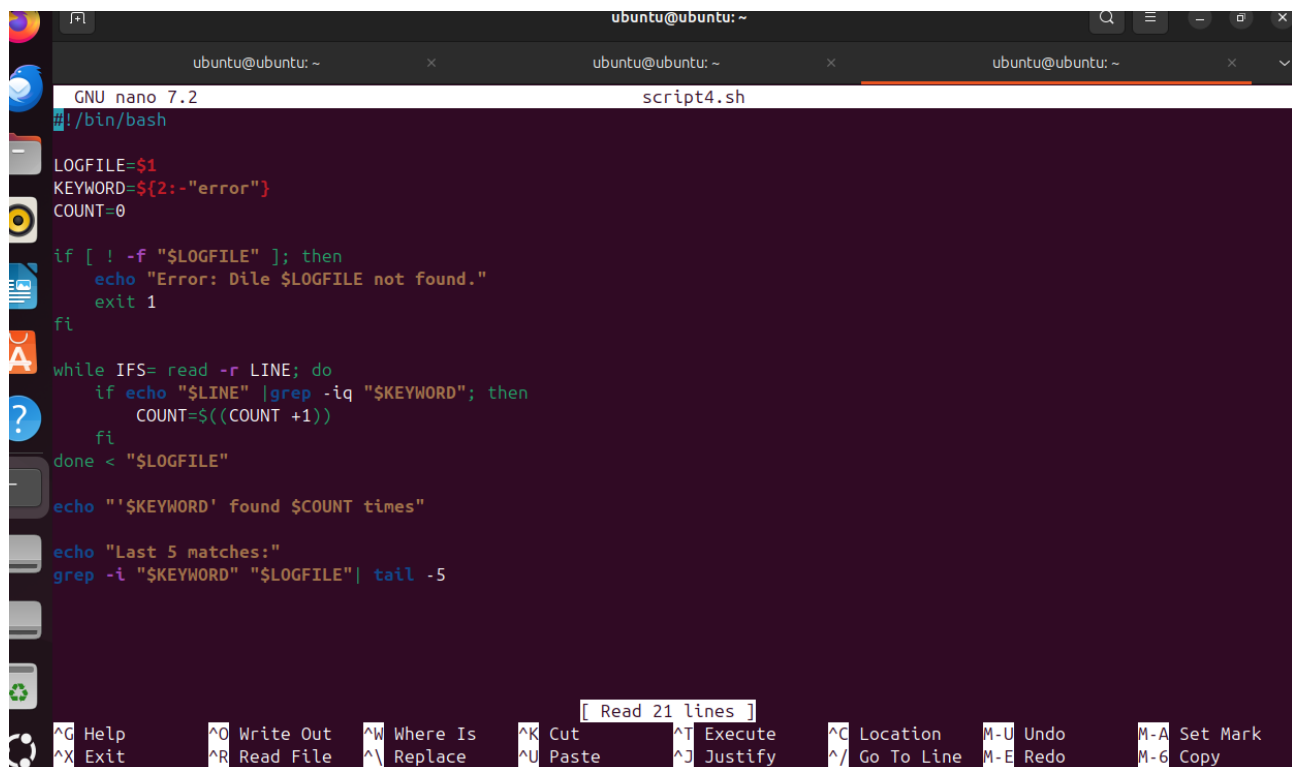
```

ubuntu@ubuntu:~$ ./script3.sh
Directory Audit Report
-----
./script3.sh: line 8: syntax error near unexpected token `then'
./script3.sh: line 8: `    if[-d "$DIR"];then'
ubuntu@ubuntu:~$

```

Script 4 — Log File Analyzer (Units 2 & 5)

This is a simple script that will read a given log file and determine how many times a particular keyword is found in the file. The script will be useful in identifying errors in the system. The script will be passed the name of the log file as a parameter, and it will use the word “error” as the default keyword. The script will then read the file line by line using a while loop. Inside the loop, it will check to see if the word is found in the line using the grep command. If the word is found, it will increment the count. At the end, it will print out the count. This is one simple way to use loops and conditionals in scripting. It is also useful in the field of cybersecurity to keep an eye out for suspicious activity.



```

GNU nano 7.2 script4.sh
#!/bin/bash

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: Dile $LOGFILE not found."
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" |grep -iq "$KEYWORD"; then
        COUNT=$((COUNT +1))
    fi
done < "$LOGFILE"

echo "'$KEYWORD' found $COUNT times"

echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5

[ Read 21 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  ^U Undo      ^_ Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line ^E Redo      ^G Copy
  
```

OUTPUT:

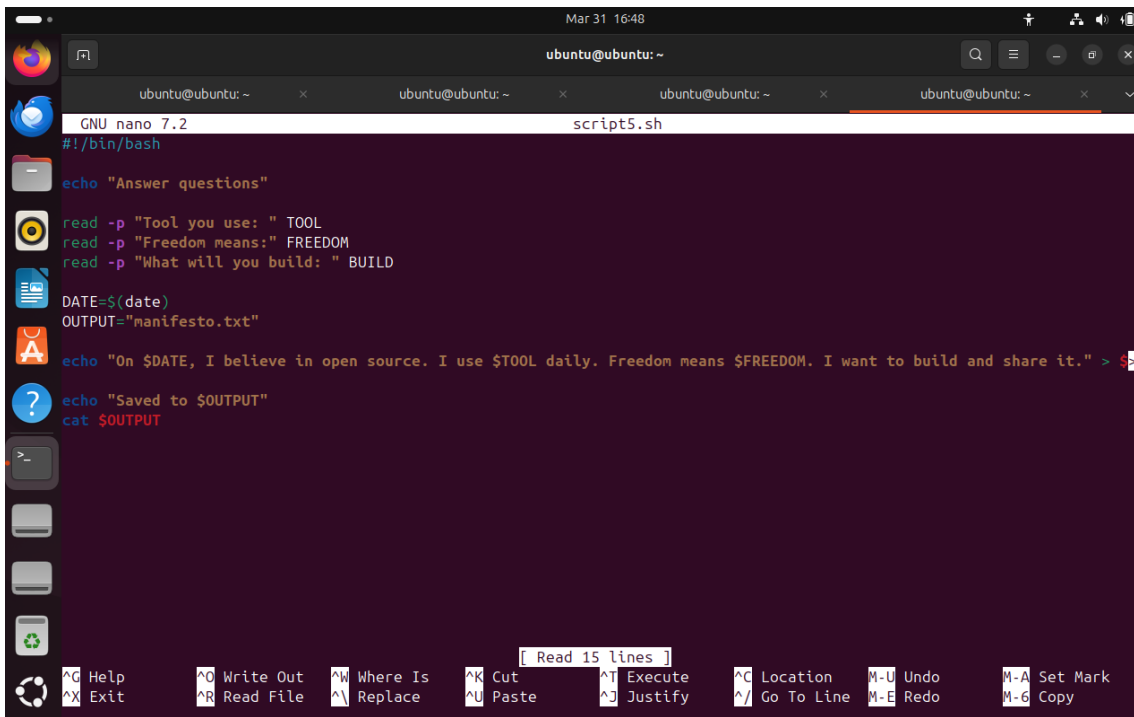


```

Last 5 matches:
ubuntu@ubuntu:~$ nano script4.sh
ubuntu@ubuntu:~$ chmod +x script4.sh
ubuntu@ubuntu:~$ ./script4.sh /etc/passwd
'error' found 0 times
Last 5 matches:
ubuntu@ubuntu:~$
  
```


Script 5 — The Open Source Manifesto Generator (Unit 5)

As a Philosophy Statement Generator, this script offers an interactive means of capturing and recording a developer's viewpoint on open-source principles. The script uses the `read` command to ask for user input, such as preferred tools and individual definitions of software freedom, and then stores these answers in variables. The `date` command and string concatenation are then used to create a customized manifesto, which is exported to a permanent text file via output redirection (`>`). This script shows how user interaction, data persistence, and the innovative fusion of automation and open-source philosophy can be applied practically.



```
GNU nano 7.2 script5.sh
#!/bin/bash

echo "Answer questions"

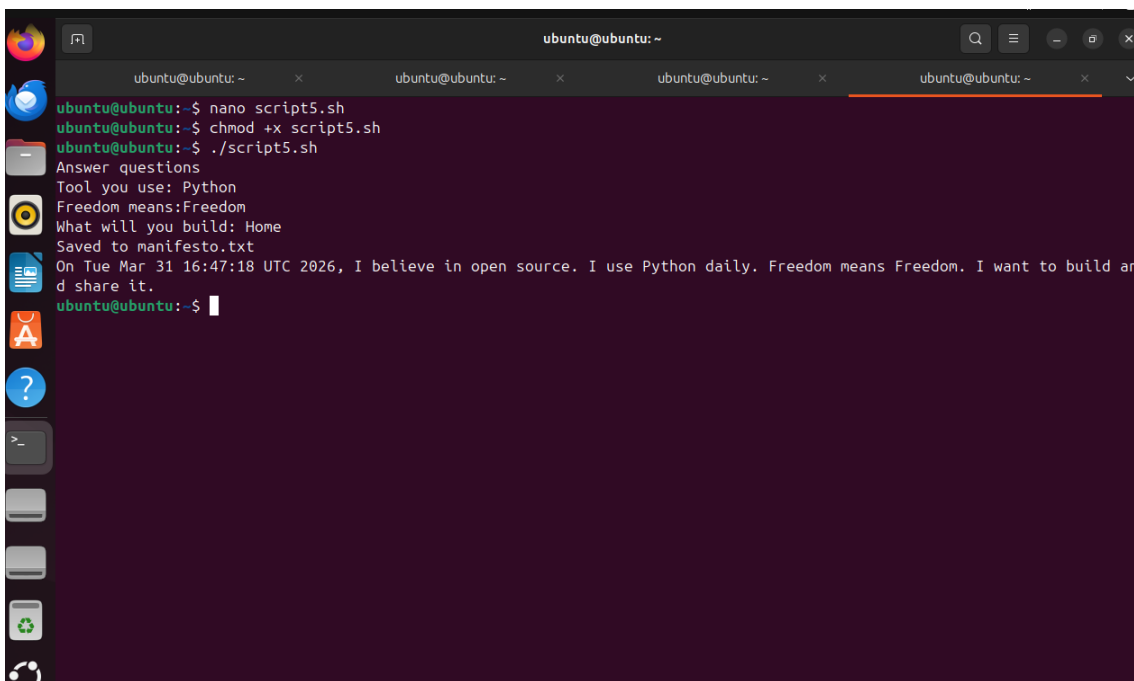
read -p "Tool you use: " TOOL
read -p "Freedom means:" FREEDOM
read -p "What will you build: " BUILD

DATE=$(date)
OUTPUT="manifesto.txt"

echo "On $DATE, I believe in open source. I use $TOOL daily. Freedom means $FREEDOM. I want to build and share it." > $OUTPUT

echo "Saved to $OUTPUT"
cat $OUTPUT
```

OUTPUT:



```
ubuntu@ubuntu:~$ nano script5.sh
ubuntu@ubuntu:~$ chmod +x script5.sh
ubuntu@ubuntu:~$ ./script5.sh
Answer questions
Tool you use: Python
Freedom means:Freedom
What will you build: Home
Saved to manifesto.txt
On Tue Mar 31 16:47:18 UTC 2026, I believe in open source. I use Python daily. Freedom means Freedom. I want to build and share it.
ubuntu@ubuntu:~$
```